

Практическая работа №3

Теоретическое введение

API (Application Programming Interface) представляет собой набор методов и соглашений, которые определяют, как различные компоненты программного обеспечения могут взаимодействовать друг с другом. Это может быть интерфейс между различными компонентами одного приложения, а также между отдельными приложениями или сервисами.

В микросервисной архитектуре, где приложение разбито на отдельные независимые сервисы, API становится ключевым механизмом взаимодействия между этими сервисами. Каждый микросервис предоставляет API для выполнения определенных функций или операций.

Существует несколько способов взаимодействия между микросервисами через их API:

1. HTTP/HTTPS запросы: Самый распространенный способ взаимодействия. Каждый микросервис предоставляет набор HTTP эндпоинтов (URL-адресов), по которым другие микросервисы могут отправлять HTTP запросы для выполнения операций.

2. RPC (Remote Procedure Call): Это способ взаимодействия, при котором вызывающий микросервис вызывает удаленный метод или процедуру непосредственно на удаленном микросервисе. Это обычно реализуется с использованием специализированных библиотек и протоколов, таких как gRPC.

3. Message Brokers: В этом случае микросервисы обмениваются сообщениями через посредника сообщений (например, RabbitMQ или Apache Kafka). Это обеспечивает асинхронное взаимодействие и возможность работы в режиме "публикация-подписка".

4. GraphQL: это язык запросов и среда выполнения для API, разработанных и выпущенных компанией Facebook. Он позволяет клиентам запрашивать только те данные, которые им нужны, и предоставляет гибкую схему для описания данных и операций.

Взаимодействие между микросервисами через их API играет ключевую роль в микросервисной архитектуре, обеспечивая гибкость, масштабируемость и отказоустойчивость системы. Однако при проектировании и разработке API необходимо учитывать множество факторов, таких как безопасность, производительность и удобство использования.

Полезные ссылки

1. Официальная документация Docker: <https://docs.docker.com/>
2. Docker Hub: <https://hub.docker.com/>
3. Docker Community Forums: <https://forums.docker.com/>
4. Микросервисная архитектура: введение и основные концепции:
<https://microservices.io/>
5. Что такое API: <https://habr.com/ru/articles/464261/>

Задание

Цель работы: внедрить API в приложение и обеспечить связь между микросервисами

Данная практическая работа направлена на изучение использования API с точки зрения микросервисов. По сути, любой микросервис представляет собой отдельный API. Именно поэтому возможна их связь и последующее внедрение в приложение.

В ходе выполнения практики могут быть использованы любые языковые средства и фреймворки, для создания микросервисов.

Давайте рассмотрим создание API на примере веб-приложения по менеджменту задач.

В приложении реализованы три микросервиса:

Сервис управления задачами (Task Service): Этот сервис отвечает за управление задачами. Он предоставляет API для создания новых задач и получения списка существующих задач. Для каждой задачи хранится описание и идентификатор пользователя, создавшего задачу.

Пример файла task_service.py:

```
# task_service.py
from flask import Flask, jsonify, request

app = Flask(__name__)

tasks = []

@app.route('/tasks', methods=['GET'])
def get_tasks():
    return jsonify(tasks)

@app.route('/tasks', methods=['POST'])
def create_task():
    data = request.get_json()
    description = data.get('description')
    if not description:
```

```

        return jsonify({'error': 'Task description is
required'}), 400
        user_id = data.get('user_id') # предполагаем, что
user_id передается в запросе
        task = {'description': description, 'user_id':
user_id}
        tasks.append(task)
        return jsonify({'message': 'Task created
successfully'}), 201

@app.route('/tasks/<int:task_id>', methods=['GET'])
def get_task(task_id):
    # Реализация получения информации о задаче с
task_id
    return jsonify({'task_id': task_id, 'description':
'Task description'})

@app.route('/tasks/<int:task_id>', methods=['PUT'])
def update_task(task_id):
    # Реализация обновления задачи с task_id
    return jsonify({'message': 'Task updated'})

@app.route('/tasks/<int:task_id>',
methods=['DELETE'])
def delete_task(task_id):
    # Реализация удаления задачи с task_id
    return jsonify({'message': 'Task deleted'})

if __name__ == '__main__':
    app.run(debug=True, port='5001', host='0.0.0.0')

```

Сервис пользователя (User Service): Этот сервис отвечает за управление пользователями. Он предоставляет API для регистрации новых пользователей. Каждый пользователь характеризуется именем пользователя и паролем.

```

# user_service.py
from flask import Flask, jsonify, request

app = Flask(__name__)

```

```

users = []

@app.route('/register', methods=['POST'])
def register():
    data = request.get_json()
    username = data.get('username')
    password = data.get('password')
    if not username or not password:
        return jsonify({'error': 'Invalid username or password'}), 400
    user = {'username': username, 'password': password}
    users.append(user)
    return jsonify({'message': 'User registered successfully'}), 201

@app.route('/login', methods=['POST'])
def login_user():
    # Реализация аутентификации пользователя
    return jsonify({'token': 'user_token'})

if __name__ == '__main__':
    app.run(debug=True, port='5002', host='0.0.0.0')

```

Веб-приложение (Web Service): это веб-приложение, которое взаимодействует с пользователем. Оно предоставляет интерфейс для регистрации новых пользователей и управления задачами. Пользователь может зарегистрироваться, просматривать список задач и создавать новые задачи.

```

from flask import Flask, render_template, request, jsonify
import requests

app = Flask(__name__)

TASK_SERVICE_URL = 'http://task_service:5001'
USER_SERVICE_URL = 'http://user_service:5002'

@app.route('/')

```

```

def index():
    return render_template('index.html')

@app.route('/tasks', methods=['GET', 'POST'])
def tasks():
    if request.method == 'GET':
        response = requests.get(f'{TASK_SERVICE_URL}/tasks')
        tasks = response.json()
        return render_template('tasks.html', tasks=tasks)
    elif request.method == 'POST':
        data = {'description': request.form['description']}
        response = requests.post(f'{TASK_SERVICE_URL}/tasks', json=data)
        return jsonify(response.json())

@app.route('/register', methods=['GET', 'POST'])
def register():
    if request.method == 'GET':
        return render_template('register.html')
    elif request.method == 'POST':
        data = {'username': request.form['username'], 'password': request.form['password']}
        response = requests.post(f'{USER_SERVICE_URL}/register', json=data)
        return jsonify(response.json())

if __name__ == '__main__':
    app.run(debug=True, port='5000', host='0.0.0.0')

```

Каждый сервис реализован как отдельное приложение на языке программирования Python с использованием фреймворка Flask. Для каждого сервиса был создан Dockerfile, который определяет окружение контейнера и запускает приложение.

Пример Docker-файла:

```
# web/Dockerfile
```

```
FROM python:3.9

WORKDIR /app

COPY requirements.txt .

RUN pip install --no-cache-dir -r requirements.txt

COPY . .

CMD ["python", "app.py"]
```

Для взаимодействия между сервисами мы использовали HTTP запросы. Например, при создании новой задачи веб-приложение отправляет запрос к сервису управления задачами, а при регистрации нового пользователя - к сервису пользователя.

Наконец, мы использовали Docker Compose для управления и развертывания наших микросервисов. Мы создали отдельный файл `docker-compose.yml`, который описывает конфигурацию и связи между нашими сервисами, позволяя нам легко развернуть всю архитектуру одной командой.

Пример Docker-compose файла:

```
version: '3'

services:
  task_service:
    build:
      context: ./task_service
    ports:
      - "5001:5001"
  user_service:
    build:
      context: ./user_service
    ports:
```



```
    - "5002:5002"
web:
  build:
    context: ./web
  ports:
    - "5000:5000"
  depends_on:
    - task_service
    - user_service
```

Вопросы к практической работе

1. Какие преимущества предоставляет использование API в микросервисной архитектуре?
2. Какие типы API чаще всего используются для взаимодействия между микросервисами и почему?
3. Какие способы аутентификации и авторизации могут быть применены при взаимодействии микросервисов через их API?
4. Какие механизмы обработки ошибок и исключений следует использовать при работе с внешними API других микросервисов?
5. Каким образом обеспечивается версионирование API в микросервисной архитектуре и почему это важно?
6. Какие инструменты и технологии можно использовать для тестирования API и взаимодействия между микросервисами?
7. Какие принципы следует соблюдать при проектировании и документировании API микросервисов?
8. Каким образом реализуется управление сетевым взаимодействием и маршрутизацией между микросервисами в контейнерной среде?
9. Какие меры безопасности следует принимать при использовании API для взаимодействия между микросервисами?

10. Какие лучшие практики следует соблюдать при мониторинге и отладке взаимодействия микросервисов через их API?

Критерии оценки

- Созданы API
- Микросервисы связаны между собой и внедрены в приложение
- Проверена работа приложения
- Составлен отчет о проделанной работе со скриншотами её выполнения
- Даны ответы на вопросы к практической работе